✦ V4 ✦

Build a Blog Writer Agent with Images & UI

Image Generation Edition — Vibe Coding Guide for Non-Tech Students

11 Steps	Gemini AI	Real Images	Streamlit	Full UI

Tool: Antigravity | APIs: OpenAI · Tavily · Google Gemini | AI PM Cohort · Ashu Mishra

What's Improved in V4 — And Why It Matters

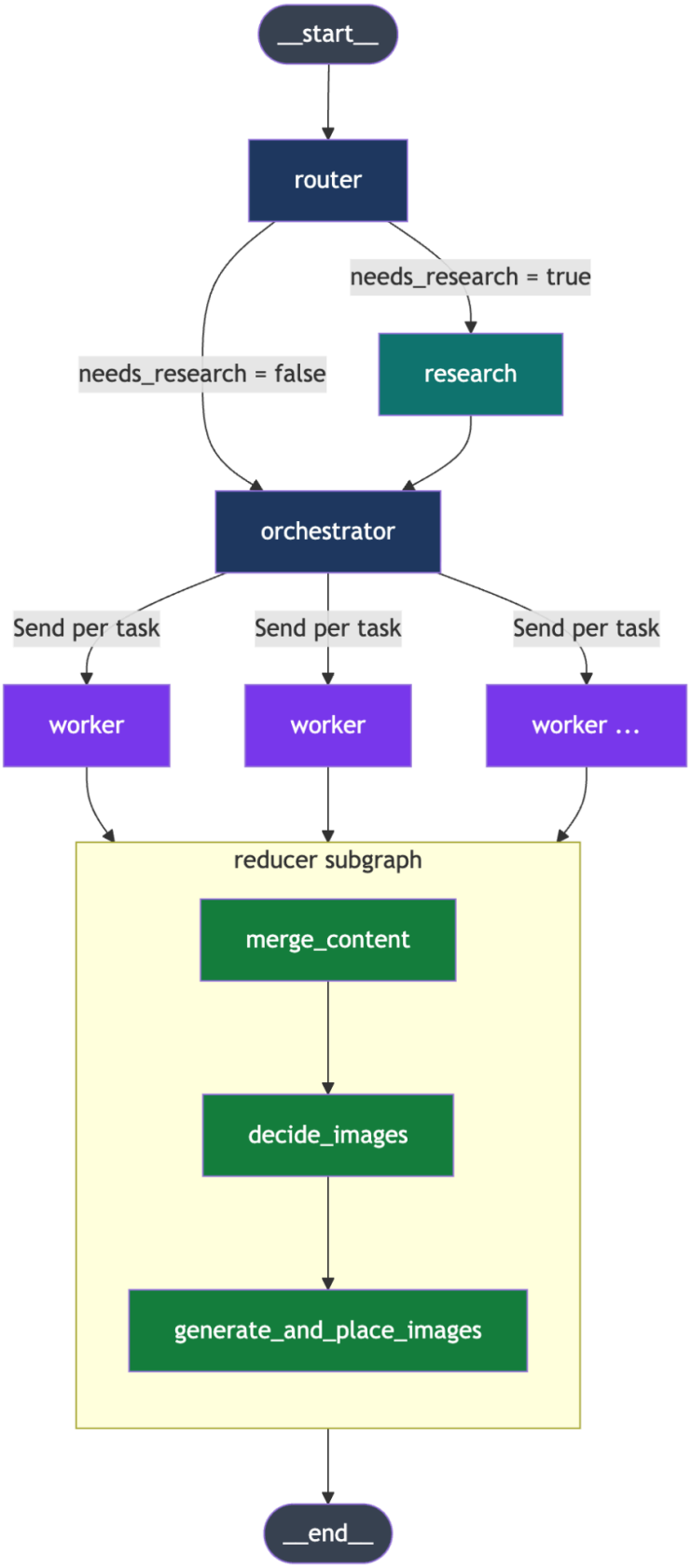
V4 adds two major new capabilities: AI-generated images embedded inside the blog, and a full Streamlit web interface. Read this before building anything.

Area	V3 — What it had	V4 — What's new	Why this makes it better
Biggest addition	Reducer was a single node	Reducer is now a 3-node subgraph: merge_content → decide_images → generate_and_place_images	Separates concerns cleanly. Each node does one job. Easy to debug, extend, or swap the image model.
Image decision node	None	decide_images: LLM reads the full blog and decides where to insert up to 3 image placeholders ([[IMAGE_1]] etc.) and writes a generation prompt for each	The LLM knows which concepts need visual explanation — it inserts images at the right points, not randomly.
Image generation	None	generate_and_place_images: calls Gemini 2.5 Flash to generate each image as bytes, saves to images/ folder, replaces placeholders with Markdown image syntax	Real AI-generated diagrams embedded directly in the blog. Not stock photos — content-aware technical visuals.
Two new data models	No image models	ImageSpec (one image: placeholder, filename, alt, caption, prompt, size, quality) + GlobalImagePlan (the full image decision for the blog)	Structured output ensures the LLM returns a machine-readable image plan, not free text.
Graceful fallback	No error handling on output	If Gemini fails (quota/safety), placeholder is replaced with a formatted error block — blog remains readable	Production requirement: a failed image should never break the whole document.
Third API key	OpenAI + Tavily	Adds GOOGLE_API_KEY for Gemini image generation	Gemini 2.5 Flash is the image model. Google API key is free to get and has a generous free tier.
Router — recency logic	No date awareness	Router now sets recency_days: open_book=7, hybrid=45, closed_book=3650. as_of date passed through the whole pipeline	Evidence is filtered by date in open_book mode — only articles from the last 7 days count as valid sources.
RouterDecision model	needs_research, mode, queries	Adds reason (why this mode was chosen) and max_results_per_query (how many Tavily results per query)	More transparency — you can see the router's reasoning, not just its decision.
Research — date filtering	No date filtering	In open_book mode, evidence items with dates	Prevents the agent from citing a 2-year-old article as 'recent news'.

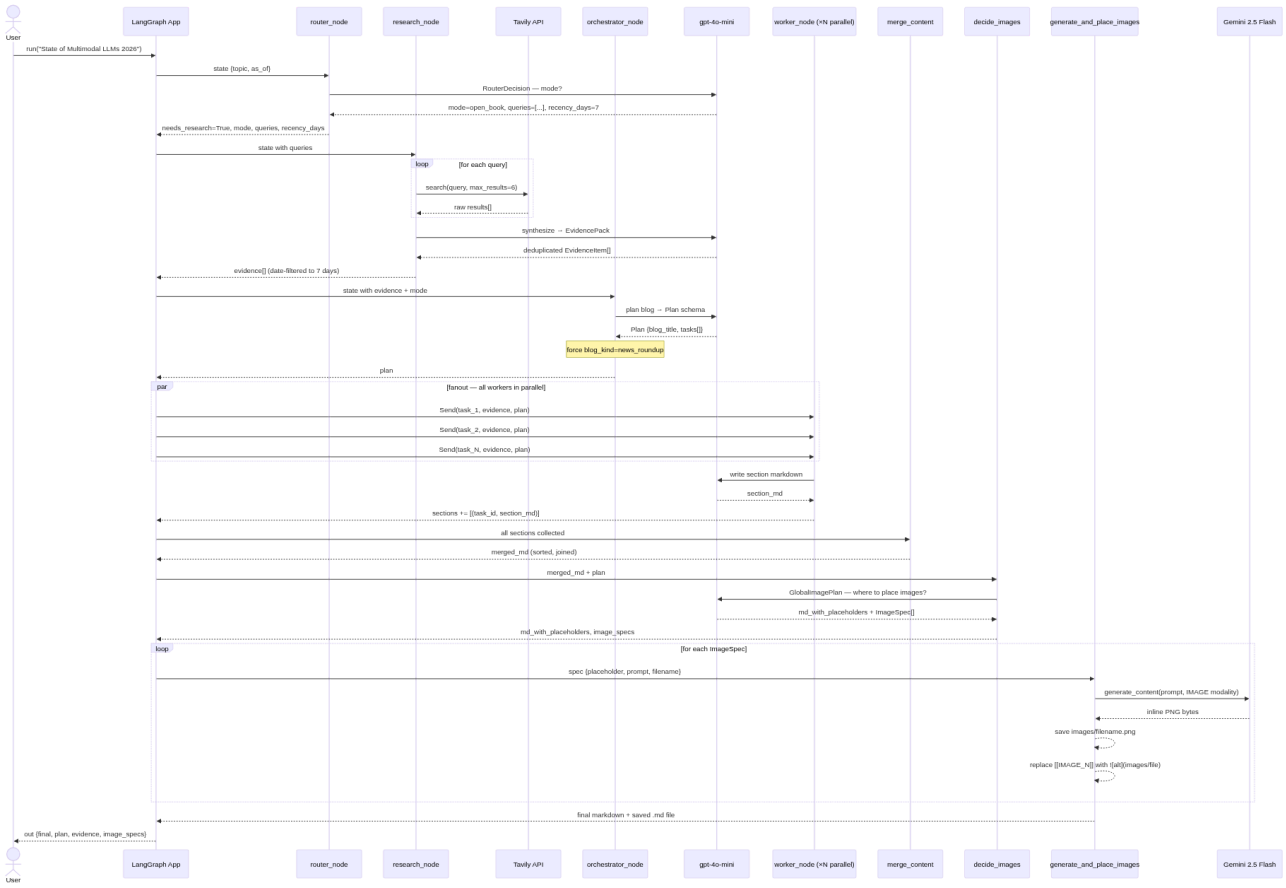
		outside the recency window are discarded	
_safe_slug helper	Raw blog title used as filename	Proper slug function: lowercase, alphanumeric + underscores only, strips special characters	Prevents file system errors from blog titles with colons, quotes, or special characters.
Streamlit frontend	No UI — notebook only	Full Streamlit app: 5 tabs (Plan, Evidence, Markdown Preview, Images, Logs), sidebar with past blogs, streaming progress, download buttons	Students can generate, preview, and download blogs through a real web interface.
State new fields	mode, needs_research, queries, evidence, plan, sections, final	Adds: as_of (today's date string), recency_days (int), merged_md, md_with_placeholders, image_specs	Each new field serves the image pipeline or the date-aware research pipeline.

The Core Insight of V4

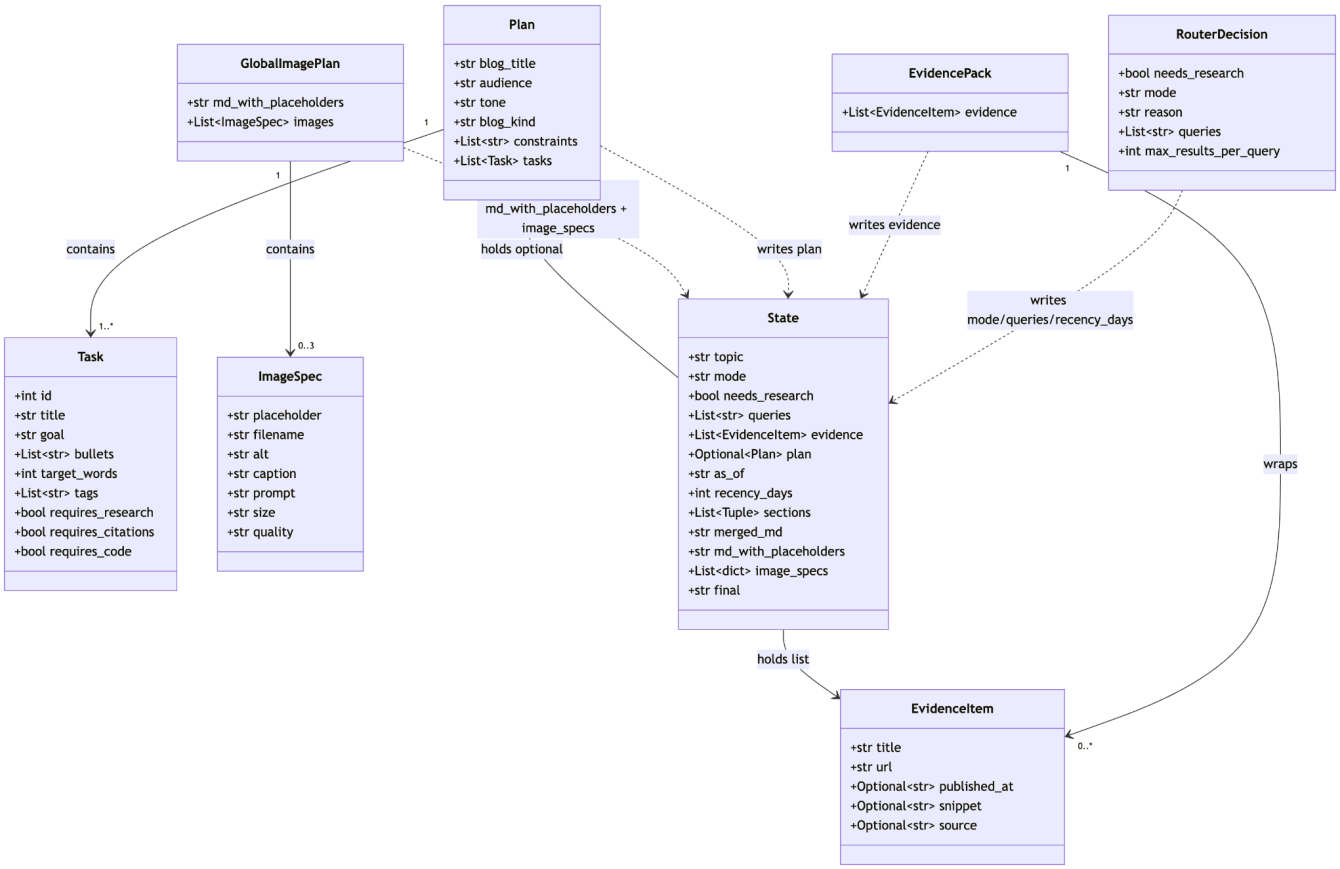
V3 produced text. V4 produces a complete publishable blog — with AI-generated diagrams placed exactly where they help understanding most, wrapped in a web UI a non-technical user can operate without touching the terminal. The architecture is the same LangGraph pattern, but the reducer is now a mini-pipeline of its own.


__start__
router
needs_research = true
needs_research = false
research
orchestrator
Send per task
Send per task
Send per task
worker
worker
worker ...
reducer subgraph
merge_content
decide_images
generate_and_place_images
__end__

Flow Chart

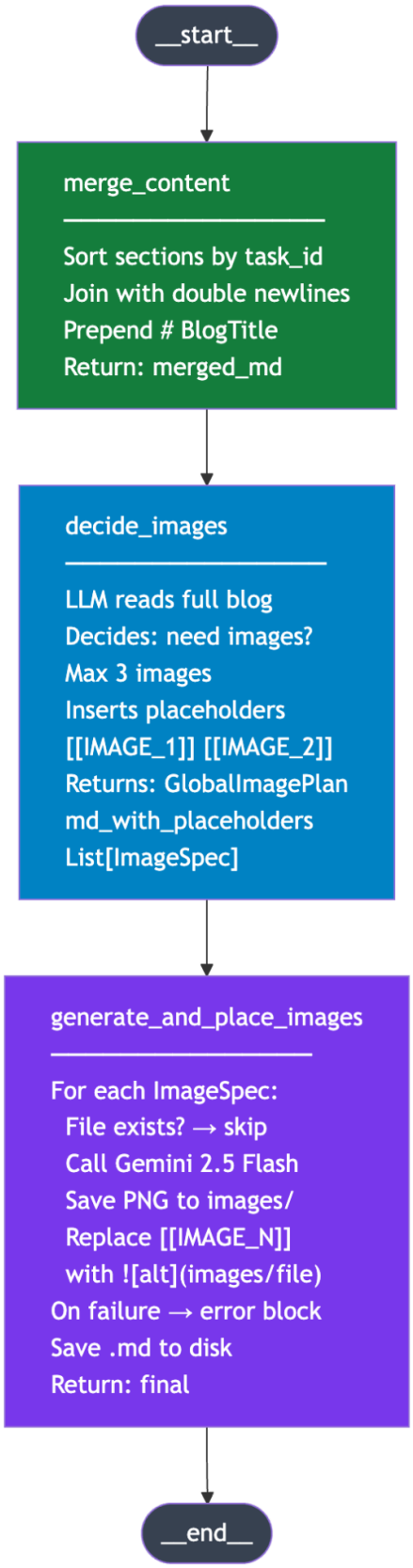


Sequence Diagram



Class Diagram

How Image Generation Works — The Reducer Subgraph


__start__
merge_content
Sort sections by task_id
Join with double newlines
Prepend # BlogTitle
Return: merged_md
decide_images
LLM reads full blog
Decides: need images?
Max 3 images
Inserts placeholders
[[IMAGE_1]] [[IMAGE_2]]
Returns: GlobalImagePlan
md_with_placeholders
List[ImageSpec]
generate_and_place_images
For each ImageSpec:
File exists? → skip
Call Gemini 2.5 Flash
Save PNG to images/
Replace [[IMAGE_N]]
with ![alt](images/file)
On failure → error block
Save .md to disk
Return: final
__end__

This is the most technically interesting part of V4. The single 'reducer' node from V3 is replaced by a 3-node subgraph. Each node does exactly one thing. Read this page carefully before building Steps 08 and 09.

Node 1 — merge_content

Sort sections by ID → Join into one Markdown document → Add H1 title

Input: List of (task_id, section_text) tuples — possibly out of order because parallel workers finish at different times

→

Output: One clean Markdown string with title, all sections in correct order, saved to merged_md in State

Node 2 — decide_images

LLM reads the full blog and decides: does this need diagrams? Where? What should they show?

The LLM gets the full blog text and asks itself:

1. Does this concept benefit from a diagram?
2. Max 3 images — which 3 help most?
3. What exact prompt generates each one?

→

Inserts placeholders into the blog:

`[[IMAGE_1]] after the intro section`

`[[IMAGE_2]] after the core concepts`

`[[IMAGE_3]] before the conclusion`

→

Returns GlobalImagePlan with:

- md_with_placeholders: the full blog text with [[IMAGE_N]] tags inserted
- images: list of ImageSpec objects (one per image)

Node 3 — generate_and_place_images

For each ImageSpec: call Gemini → save PNG → replace [[IMAGE_N]] with ![alt](images/file.png)

📤 **Send prompt**
ImageSpec.prompt is sent to Gemini 2.5 Flash with response_modalities=[IMAGE]

→

📥 **Receive bytes**
Gemini returns raw PNG bytes via inline_data. Extracted from resp.candidates[0].content.parts

→

💾 **Save to disk**
Written to images/filename.png. Skipped if file already exists (no duplicate generation).

→

📝 **Replace placeholder**
`[[IMAGE_1]] → ![alt text](images/file.png) + italic caption on next line`

⚠️ What happens if Gemini fails?

If image generation fails for any reason (quota exceeded, safety filter, network error), the node does NOT crash the whole blog. Instead, it replaces the [[IMAGE_N]] placeholder with a formatted error block that shows the caption, alt text, and the original generation prompt — so the blog is still fully readable. This is called graceful fallback and it is a production requirement.

📄 Final Output

A .md file saved to disk with the blog title as filename (slugified). All [[IMAGE_N]] placeholders replaced. Images saved in images/ subfolder. Both file and folder ready to publish.

How to Use This Guide

This is a vibe coding guide. You describe what you want in plain English. Antigravity writes the code. The only exception is Step 11 — the frontend code must be used exactly as provided.

Step 1	Read 'What's happening here' and study the image generation flowchart before building Steps 08–09.
Step 2	Create two files: bwa_backend.py and bwa_frontend.py. They must be in the same folder.
Step 3	Your .env file needs THREE keys: OPENAI_API_KEY, TAVILY_API_KEY, and GOOGLE_API_KEY. See the last page for how to get the Google key.
Step 4	Copy the prompt from the dark box into Antigravity. For Step 11 (frontend), also copy the Reference Code box exactly into bwa_frontend.py.
Step 5	Check the 'What you should see' list. If something is wrong, describe the problem to Antigravity in plain English.

🛠️

STEP 01

Project Setup & Three API Keys

Three APIs, one new library — understand Google Gemini before you install it

What's happening here?

V4 needs three API keys — not two. You already have OpenAI and Tavily from V3. The new one is GOOGLE_API_KEY, used by Gemini 2.5 Flash to generate images.

One new Python library is also required: google-genai. This is Google's official SDK for their Gemini AI models. It is different from the LangChain Google integrations — this one talks directly to Google's API and lets you request image output alongside text.

The backend lives in a file called bwa_backend.py and the frontend in bwa_frontend.py. Both files must be in the same folder. The frontend imports the compiled LangGraph app directly from the backend — so the backend must be complete and error-free before the frontend can run.

You also need to install two extra packages beyond V3: google-genai for Gemini image generation, and streamlit + pandas for the frontend UI.

⚡ What's improved vs V3 in this step

- New: google-genai library for Gemini 2.5 Flash image generation
- New: streamlit and pandas for the frontend UI
- Three API keys now: OPENAI_API_KEY, TAVILY_API_KEY, GOOGLE_API_KEY
- Two files instead of one: bwa_backend.py and bwa_frontend.py

YOUR PROMPT — Copy and paste into Antigravity

```
I want to build V4 of my Blog Writer Agent. This version generates real AI images
inside the blog and has a Streamlit web UI.

Please create two empty Python files: bwa_backend.py and bwa_frontend.py in the
same folder.

Create a .env file with three API key placeholders:
- OPENAI_API_KEY=your_openai_key_here
- TAVILY_API_KEY=your_tavily_key_here
- GOOGLE_API_KEY=your_google_api_key_here

Install all required packages for V4. The new ones compared to V3 are:
- google-genai (Google's SDK for Gemini AI — used for image generation)
- streamlit (web UI framework)
- pandas (for displaying tables in the UI)

Also keep all the V3 packages: langgraph, langchain-openai, langchain-community,
pydantic, python-dotenv.

After setup, explain: what is Gemini 2.5 Flash and how is it different from using
DALL-E or Stable Diffusion for image generation?
```

✅ **What you should see after this step**

- Two empty Python files: bwa_backend.py and bwa_frontend.py
- .env file with three placeholders
- google-genai, streamlit, and pandas installed successfully
- An explanation from Antigravity about Gemini 2.5 Flash vs other image models

STEP 02

All Data Models — Two New Image Models

ImageSpec and GlobalImagePlan — the structured blueprint for AI image generation

What's happening here?

V4 has all the same models as V3 (Task, Plan, EvidenceItem, RouterDecision, EvidencePack, State) plus two brand new ones for the image pipeline.

ImageSpec represents one image to generate. It contains everything needed: placeholder (the [[IMAGE_1]] tag that gets inserted into the blog), filename (where to save the image on disk, e.g. qkv_flow.png inside an images/ folder), alt text (for accessibility), a caption (shown under the image in the blog), the actual prompt to send to Gemini, the image size (square, portrait, or landscape), and quality (low/medium/high — affects cost and detail).

GlobalImagePlan is the complete image decision for the whole blog. It contains the full markdown text with [[IMAGE_N]] placeholders already inserted at the right spots, plus a list of ImageSpec objects — one for each placeholder. When the LLM returns a GlobalImagePlan, you immediately know exactly where the images go and what to generate for each one.

The State also gains several new fields: as_of (today's date as a string), recency_days (how far back evidence is considered valid), merged_md (the fully joined blog text after parallel workers finish), md_with_placeholders (same text but with image placeholders inserted), and image_specs (the list of image spec dictionaries ready for generation).

⚡ What's improved vs V3 in this step

- New: ImageSpec model — placeholder, filename, alt, caption, prompt, size, quality
- New: GlobalImagePlan model — md_with_placeholders + list of ImageSpec objects
- State gains: as_of, recency_days, merged_md, md_with_placeholders, image_specs
- RouterDecision gains: reason (why this mode) + max_results_per_query

YOUR PROMPT — Copy and paste into Antigravity

```
In bwa_backend.py, add all the data models for the V4 Blog Writer Agent. Build them
in this order:

Task (Pydantic): id (int), title (str), goal (str — one sentence), bullets (list of
str, min 3 max 6), target_words (int 120-550), tags (list of str, default empty),
requires_research (bool, default False), requires_citations (bool, default False),
requires_code (bool, default False).

Plan (Pydantic): blog_title, audience, tone, blog_kind (Literal:
explainer/tutorial/news_roundup/comparison/system_design, default explainer),
constraints (list of str, default empty), tasks (list of Task).

EvidenceItem (Pydantic): title, url, published_at (optional str — ISO date only if
explicitly known), snippet (optional str), source (optional str).

RouterDecision (Pydantic): needs_research (bool), mode (Literal:
closed_book/hybrid/open_book), reason (str — why this mode was chosen), queries
(list of str, default empty), max_results_per_query (int, default 5).

EvidencePack (Pydantic): evidence (list of EvidenceItem, default empty).
```

```
ImageSpec (Pydantic — NEW):
- placeholder: str (e.g. [[IMAGE_1]])
- filename: str (save under images/ folder, e.g. qkv_flow.png)
- alt: str (accessibility text)
- caption: str (shown under the image in the blog)
- prompt: str (the exact prompt to send to the image model)
- size: Literal with options "1024x1024", "1024x1536", "1536x1024" — default
"1024x1024"
- quality: Literal with options "low", "medium", "high" — default "medium"

GlobalImagePlan (Pydantic — NEW):
- md_with_placeholders: str (the full blog markdown with [[IMAGE_N]] tags inserted
at the right places)
- images: list of ImageSpec objects (default empty)

State (TypedDict): topic, mode (str), needs_research (bool), queries (list of str),
evidence (list of EvidenceItem), plan (optional Plan), as_of (str — today's date),
recency_days (int), sections (list of tuple[int, str] with operator.add reducer),
merged_md (str), md_with_placeholders (str), image_specs (list of dict), final
(str).

Also initialise: llm = ChatOpenAI(model="gpt-4o-mini") and load_dotenv().

After writing the models, explain: why does ImageSpec include both a prompt field
and a filename field? What would break if the filename were auto-generated instead
of planned by the LLM?
```

✅ What you should see after this step

- All 9 models written in bwa_backend.py: Task, Plan, EvidenceItem, RouterDecision, EvidencePack, ImageSpec, GlobalImagePlan, State (TypedDict), plus llm
- ImageSpec has placeholder, filename, alt, caption, prompt, size, quality
- GlobalImagePlan has md_with_placeholders and images list
- State has all new fields: as_of, recency_days, merged_md, md_with_placeholders, image_specs
- RouterDecision has reason and max_results_per_query

🗺️

STEP 03

Router Node — Now Date-Aware

Same three modes, but now recency_days flows through the whole pipeline

What's happening here?

The Router in V4 works the same way as V3 — it classifies the topic into closed_book, hybrid, or open_book. But it gains two important new behaviours.

First, the human message now passes the as_of date (today's date). This lets the router understand time-sensitive requests better — if someone asks 'what happened in AI this week', the router knows when 'this week' is.

Second, after deciding the mode, the router sets recency_days — how far back in time evidence is considered valid. open_book gets 7 days (only last week's news counts), hybrid gets 45 days (last month and a half of developments), and closed_book gets 3650 days (10 years — effectively no date filter for evergreen content).

This recency_days value flows all the way through the pipeline into the research node, which uses it to filter out stale evidence in open_book mode. The route_next function is identical to V3 — it returns 'research' or 'orchestrator' based on needs_research.

⚡ What's improved vs V3 in this step

- as_of date passed in the human message — router is time-aware
- recency_days set by mode: open_book=7, hybrid=45, closed_book=3650
- recency_days returned to State and flows into research node
- RouterDecision now returns reason — visible in logs why mode was chosen

YOUR PROMPT — Copy and paste into Antigravity

```
In bwa_backend.py, add the Router node and its routing function.

The router_node function should:
- Use llm.with_structured_output(RouterDecision) for the decision
- Use a system prompt explaining the three modes: closed_book (evergreen concepts,
no research needed), hybrid (mostly evergreen but needs up-to-date examples/tools),
open_book (volatile: news, weekly roundups, latest rankings, pricing)
- If needs_research is True, the system prompt should instruct the AI to output
3-10 specific scoped queries. For open_book weekly topics, include queries
reflecting the last 7 days
- The human message passes the topic AND the as_of date from state
- After getting the decision, set recency_days based on mode: if open_book set 7,
if hybrid set 45, if closed_book set 3650
- Return needs_research, mode, queries, and recency_days to state

Also create a route_next function that:
- Returns the string "research" if state["needs_research"] is True
- Returns the string "orchestrator" if False

After writing both functions, explain: why does open_book mode use recency_days=7
instead of just taking all available evidence? What problem does the 7-day cutoff
solve?
```

✅ **What you should see after this step**

- A router_node function using llm.with_structured_output(RouterDecision)
- Human message includes both topic and as_of date
- recency_days set with if/elif/else: open_book=7, hybrid=45, closed_book=3650
- Function returns needs_research, mode, queries, and recency_days
- A route_next function returning 'research' or 'orchestrator'
- Antigravity's explanation of why a recency cutoff prevents stale evidence

🔍

STEP 04

Research Node — Date-Filtered Evidence

Same Tavily search, but now stale articles are filtered out in open_book mode

What's happening here?

The research node in V4 is structurally identical to V3 — it runs Tavily searches, collects raw results, uses the LLM to clean and deduplicate them into EvidenceItem objects. The core logic hasn't changed.

What's new is a date filtering step at the end, only for open_book mode. After the LLM returns the cleaned evidence pack, the code checks each item's published_at date. Any article older than the recency_days cutoff is removed from the evidence list.

For example: if the topic is 'AI news this week' and today is July 15th, recency_days=7, then only articles published on or after July 8th are kept. An article from June 1st would be filtered out even if it's otherwise relevant — because in a weekly news roundup, last month's news is stale.

The research system prompt also now receives as_of and recency_days as context, so the LLM understands the time window when synthesizing evidence.

There's also a safety check at the top: if TAVILY_API_KEY is not set in the environment, the function returns an empty list immediately instead of crashing. This makes it safe to run the agent even without a Tavily key — it will just fall back to closed_book behaviour.

⚡ What's improved vs V3 in this step

- Safety check: returns empty list immediately if TAVILY_API_KEY is not set
- as_of and recency_days passed to the research synthesizer LLM
- Date filtering: open_book evidence filtered to recency_days window after LLM cleaning
- _iso_to_date helper converts date strings safely for comparison

YOUR PROMPT — Copy and paste into Antigravity

```
In bwa_backend.py, add the Research node and its Tavily helper.

Create a private helper function _tavily_search(query, max_results=5) that:
- Immediately returns an empty list if TAVILY_API_KEY is not set in the environment
(use os.getenv)
- Imports TavilySearchResults inside a try/except block and returns empty list on
any exception
- Calls tool.invoke with the query and normalises each result to: title, url,
snippet (from content or snippet field), published_at (from published_date or
published_at), source
- Returns a list of normalised dictionaries

Create a helper function _iso_to_date(s) that:
- Takes an optional string, returns None if empty or None
- Tries to parse the first 10 characters as a date using date.fromisoformat
- Returns None on any exception (never crashes)

Create the research_node function that:
- Gets the first 10 queries from state
- Loops through them calling _tavily_search with max_results=6 and collects all raw
results
- If no raw results, returns {"evidence": []}
```

```
- Uses llm.with_structured_output(EvidencePack) with a research synthesizer system
prompt
- The system prompt tells the LLM to: only include items with non-empty URL, prefer
authoritative sources, normalise published_at to YYYY-MM-DD if reliably inferable
(else null, never guess), keep snippets short, deduplicate by URL
- The human message passes as_of, recency_days, and the raw results
- After the LLM returns the pack, deduplicates by URL using a Python dict
- If mode is open_book: filters the evidence list to only keep items where
published_at date is within recency_days of as_of date (use _iso_to_date to parse
dates, skip items where date is None or too old)
- Returns {"evidence": filtered_and_deduped_list}

After writing this, explain: what happens to a blog about 'AI news this week' if we
skip the date filter and include a 6-month-old article as evidence?
```

✅ **What you should see after this step**

- A _tavily_search helper with safety check for missing TAVILY_API_KEY
- A _iso_to_date helper that never crashes
- A research_node that loops through queries and builds raw results
- LLM called with as_of and recency_days in the human message
- Date filtering applied after LLM cleaning — only for open_book mode
- Antigravity's explanation of why stale evidence corrupts news roundups

🧠

STEP 05

Orchestrator + Fanout — Date-Aware Planning

as_of and recency_days now flow through planning and into every worker

What's happening here?

The Orchestrator in V4 is almost identical to V3. Same system prompt, same three-mode behaviour, same structured output into a Plan object. The only meaningful change is that as_of and recency_days are now passed in the human message, giving the planner full temporal context.

There is also one extra step after the LLM returns the plan: if mode is open_book, the code forces plan.blog_kind = 'news_roundup' regardless of what the LLM decided. This is an explicit override — a belt-and-suspenders approach to prevent the orchestrator from accidentally planning a tutorial when the user asked for a news roundup.

The fanout function gains two extra fields in each Send() payload: as_of and recency_days. These flow through to every worker so workers also know the temporal context when writing — particularly relevant for hybrid and open_book sections that make claims about 'current' or 'recent' tools.

⚡ What's improved vs V3 in this step

- as_of and recency_days passed in orchestrator's human message
- Explicit blog_kind='news_roundup' override for open_book mode after LLM returns
- fanout sends as_of and recency_days to every worker in the payload

YOUR PROMPT — Copy and paste into Antigravity

```
In bwa_backend.py, add the Orchestrator node and the Fanout function.

The orchestrator_node function should:
- Use llm.with_structured_output(Plan) for the plan
- Get mode and evidence from state (with safe defaults)
- Use a detailed system prompt with three-mode grounding rules:
  * closed_book: keep content evergreen, no evidence dependence
  * hybrid: use evidence for up-to-date examples, mark those tasks with
requires_research=True and requires_citations=True
  * open_book: set blog_kind to "news_roundup", no tutorial content unless
requested, if evidence is weak the plan should reflect that transparently
- The human message passes: topic, mode, as_of, recency_days, and evidence (up to
16 items as model_dump())
- After the LLM returns the plan, if mode is "open_book" force plan.blog_kind =
"news_roundup" as an explicit override
- Return {"plan": plan}

The fanout function should:
- Assert that state["plan"] is not None
- Use LangGraph's Send() to launch one worker per task simultaneously
- Each Send() passes: task as model_dump(), topic, mode, as_of, recency_days, plan
as model_dump(), evidence as list of model_dump() objects

After writing both, explain: why does the code force blog_kind='news_roundup' AFTER
the LLM returns the plan instead of just telling the LLM to do it? What could go
wrong without this override?
```

✅ **What you should see after this step**

- An orchestrator_node using llm.with_structured_output(Plan)
- Human message includes topic, mode, as_of, recency_days, and evidence
- Explicit plan.blog_kind override for open_book after LLM returns
- fanout sends as_of and recency_days in each Send() payload
- Antigravity's explanation of why an explicit override is safer than relying on the prompt

✍️

STEP 06

Worker Node — Full Temporal Context

Workers now know the date — relevant for hybrid and open_book sections

What's happening here?

The Worker node in V4 is structurally identical to V3. Same system prompt, same grounding policy, same scope guard, same code rule. It writes one section of the blog per worker, in parallel.

The only additions are temporal: the worker now receives as_of and recency_days in its payload and passes them into the human message. This gives the worker awareness of the current date when making claims like 'as of today' or 'the latest version'.

The system prompt is essentially unchanged from V3 — the same quality bar (cover all bullets, stay within word count, cite evidence in open_book, include code if required, no tutorial drift in news_roundup mode). The temporal context is purely informational — it helps the worker avoid making outdated claims in hybrid mode.

⚡ What's improved vs V3 in this step

- as_of and recency_days received in payload and passed to human message
- Worker aware of 'current date' when making temporal claims
- System prompt unchanged from V3 — same quality bar and grounding policy

YOUR PROMPT — Copy and paste into Antigravity

```
In bwa_backend.py, add the Worker node.

The worker_node function receives a payload dict and should:
- Reconstruct objects: Task(**payload["task"]), Plan(**payload["plan"]), list of
EvidenceItem objects from payload["evidence"]
- Get topic, mode, as_of, and recency_days from payload
- Format bullets as a bullet list string
- Format evidence as: one item per line with title | url | published_at (or
"date:unknown")
- Use a detailed system prompt telling the AI it is a senior technical writer
writing ONE section in Markdown
- The system prompt must include:
  * Hard constraints: cover all bullets in order, stay within ±15% of target word
count, output only section content starting with ## heading
  * Scope guard: if blog_kind is "news_roundup" do NOT write tutorials — focus on
events and implications
  * Grounding policy: if mode is "open_book", every event/company/model claim must
have a citation URL from the evidence list using ([Source](URL)) format — if not in
evidence write "Not found in provided sources"
  * Citation rule: if requires_citations is True, cite evidence URLs for external
claims
  * Code rule: if requires_code is True, include at least one minimal code snippet
  * Style: short paragraphs, bullets where helpful, code fences for code, no fluff
- The human message passes: blog title, audience, tone, blog_kind, constraints,
topic, mode, as_of, recency_days, section title, goal, target_words, tags, all
three boolean flags, bullets, and evidence text
- Returns {"sections": [(task.id, section_md)]}
```

```
After writing this, explain: why does the worker need to know as_of at all — it's a
writer, not a researcher. What is the risk of a worker that doesn't know the
current date in hybrid mode?
```

✅ **What you should see after this step**

- A worker_node function that reconstructs Task, Plan, EvidenceItems from payload
- as_of and recency_days extracted from payload and passed to human message
- System prompt has scope guard, grounding policy, citation rule, code rule
- Returns {'sections': [(task.id, section_md)]} — tuple format
- Antigravity's explanation of temporal drift risk in hybrid mode workers

STEP 07

Reducer Subgraph — Node 1: merge_content

Sort sections by ID, join into one clean Markdown document

What's happening here?

This is the first of three nodes in the reducer subgraph. Its job is simple: take all the section outputs from the parallel workers and assemble them into one clean Markdown document.

Because parallel workers finish in random order, sections arrive as a list of (task_id, section_markdown) tuples. merge_content sorts this list by task_id, extracts just the markdown text, joins them with double newlines, and prepends the blog title as a Markdown H1 heading.

The result is stored in State as merged_md — a complete, correctly-ordered blog post in plain Markdown, with no images yet. This clean version flows into the next node.

Why is this a separate node instead of part of a bigger function? Because each node in LangGraph does one job. merge_content is purely about ordering and joining text. The image decisions happen in the next node. Keeping them separate makes each node easier to test, debug, and replace.

⚡ What's improved vs V3 in this step

- V3: reducer was one function doing everything
- V4: split into 3 focused nodes — merge_content is purely about text assembly
- Output goes to merged_md in State, not directly to file
- Clean separation: merge first, decide images second, generate images third

YOUR PROMPT — Copy and paste into Antigravity

```
In bwa_backend.py, add the first node of the reducer subgraph: merge_content.

The merge_content function should:
- Get the plan from state — if plan is None, raise a ValueError with a clear
message
- Get sections from state — these are (task_id, section_markdown) tuples
- Sort the sections by task_id (the first element of each tuple)
- Extract just the markdown text (second element) from each sorted tuple
- Join all sections with double newlines
- Create final_md as: # BlogTitle on the first line, then a blank line, then the
joined body
- Store this in merged_md and return {"merged_md": merged_md}

Then build the reducer subgraph structure (we will add more nodes in the next
steps):
- Create a new StateGraph using the same State TypedDict
- Add merge_content as a node called "merge_content"
- We will add the other two nodes in Steps 08 and 09

After writing this, explain with a concrete example: if workers finish in this
order — task 3 first, then task 1, then task 5, then task 2, then task 4 — show
what the sections list looks like before and after sorting. What would the blog
read like if we skipped the sort?
```

✅ **What you should see after this step**

- A merge_content function that sorts sections by task_id
- Returns {'merged_md': merged_md} with title + ordered body
- A reducer_graph StateGraph with merge_content added as a node
- Antigravity's concrete example showing before/after sort with out-of-order sections

🎨

STEP 08

Reducer Subgraph — Node 2: decide_images

The LLM reads the full blog and decides where images would help most

What's happening here?

This is the most creative node in the whole pipeline. decide_images receives the fully-assembled blog text and asks the LLM to act as a technical editor: where in this blog would an image or diagram genuinely improve understanding?

The LLM is constrained to a maximum of 3 images and must follow strict rules: images must materially improve comprehension (not be decorative), prefer technical diagrams over illustrations, and insert exactly formatted placeholders: [[IMAGE_1]], [[IMAGE_2]], [[IMAGE_3]].

For each image, the LLM also writes a generation prompt — the exact text to send to Gemini to create that image. It chooses where in the blog each placeholder goes, what the image should show, its alt text, and its caption.

If no images are needed (e.g. for a pure news roundup where diagrams wouldn't help), the LLM returns the blog text unchanged and an empty images list. The pipeline handles both cases gracefully.

The output is a GlobalImagePlan stored in State: md_with_placeholders (the blog text with [[IMAGE_N]] tags inserted at strategic points) and image_specs (a list of image spec dictionaries ready for generation).

⚡ What's improved vs V3 in this step

- Brand new node — did not exist in V3
- LLM acts as technical editor — decides IF images are needed, not just generates them
- Max 3 images enforced by the system prompt
- Each ImageSpec includes a generation prompt written by the LLM
- Graceful zero-image case: blog text returned unchanged with empty images list

YOUR PROMPT — Copy and paste into Antigravity

```
In bwa_backend.py, add the second node of the reducer subgraph: decide_images.

The decide_images function should:
- Get merged_md and plan from state
- Use llm.with_structured_output(GlobalImagePlan) for the image planning decision
- Use a system prompt that tells the LLM it is an expert technical editor deciding
if images are needed
- The system prompt must specify:
  * Maximum 3 images total
  * Each image must materially improve understanding — prefer diagrams, flows, or
table-like visuals
  * No decorative images
  * Insert placeholders in this exact format: [[IMAGE_1]], [[IMAGE_2]], [[IMAGE_3]]
  * If no images are needed, md_with_placeholders must equal the input text
unchanged and images must be an empty list
  * Output must strictly match the GlobalImagePlan schema
- The human message passes: blog_kind, topic, an instruction to insert placeholders
and propose image prompts, and the full merged_md text
- Return {"md_with_placeholders": image_plan.md_with_placeholders, "image_specs":
list of model_dump() for each image}
```

```
Then add decide_images to the reducer subgraph and connect it:
- Add an edge from "merge_content" to "decide_images"

After writing this, explain: what would happen if the LLM decided to insert 10
images instead of 3? How does the system prompt constraint prevent this, and what
would you do if you wanted to change the limit to 5?
```

✅ **What you should see after this step**

- A decide_images function using llm.with_structured_output(GlobalImagePlan)
- System prompt enforces max 3 images with strict [[IMAGE_N]] placeholder format
- Human message includes blog_kind, topic, and the full merged_md
- Returns md_with_placeholders and image_specs as list of dicts
- decide_images added to reducer_graph with edge from merge_content
- Antigravity's explanation of how to change the image limit

STEP 09

Reducer Subgraph — Node 3: generate_and_place_images

Gemini generates each image, saves to disk, replaces placeholders — with graceful fallback

What's happening here?

This is the final node in the reducer subgraph. It loops through each ImageSpec, calls Gemini 2.5 Flash to generate the image, saves it as a PNG file in an images/ folder, and replaces the [[IMAGE_N]] placeholder in the blog text with a proper Markdown image tag.

The Gemini call uses a private helper _gemini_generate_image_bytes. It creates a Google genai client using GOOGLE_API_KEY, calls models.generate_content with response_modalities=['IMAGE'], and extracts the raw PNG bytes from the response parts. The SDK has slightly different structures depending on the version, so the code checks two possible locations for the parts.

An important optimisation: if the image file already exists on disk, it is NOT regenerated. This means if you run the agent twice with the same blog, you won't waste API calls re-generating the same images.

If generation fails for any reason (quota, safety filter, network error), the code catches the exception and replaces the placeholder with a formatted error block — the blog title, alt text, original prompt, and error message displayed as a Markdown blockquote. The blog remains fully readable.

After all images are processed, the node saves the final .md file to disk using a safe slug of the blog title, and returns the final markdown string in State.

⚡ What's improved vs V3 in this step

- Brand new node — did not exist in V3
- Gemini 2.5 Flash called via google-genai SDK for image generation
- Images saved to images/ folder — only generated if file doesn't exist yet
- Graceful fallback: failed images replaced with formatted error block
- _safe_slug helper used for filenames (no special characters)
- Final .md file saved with slugified title

YOUR PROMPT — Copy and paste into Antigravity

```
In bwa_backend.py, add the third node of the reducer subgraph:
generate_and_place_images, plus its Gemini helper and slug helper.

First, create a private helper _gemini_generate_image_bytes(prompt) that:
- Imports google.genai and google.genai.types inside the function
- Gets GOOGLE_API_KEY from os.environ — raises RuntimeError if not set
- Creates a genai.Client with the API key
- Calls client.models.generate_content with model "gemini-2.5-flash-image", passing
the prompt as contents
- Sets config with response_modalities=["IMAGE"] and a safety setting for
HARM_CATEGORY_DANGEROUS_CONTENT with threshold BLOCK_ONLY_HIGH
- Extracts parts from the response: first try resp.parts, then try
resp.candidates[0].content.parts
- Raises RuntimeError if no parts found
- Loops through parts, finds the first one with inline_data that has data, and
returns that data as bytes
- Raises RuntimeError if no inline image bytes found

Create a _safe_slug(title) helper that:
```

```
- Strips and lowercases the title
- Uses re.sub to remove anything that is not alphanumeric, space, underscore, or
hyphen
- Replaces spaces with underscores
- Returns the result or "blog" if empty

Create the generate_and_place_images function that:
- Gets plan, md_with_placeholders (or falls back to merged_md), and image_specs
from state
- If image_specs is empty: saves md directly to disk as _safe_slug(blog_title).md
and returns {"final": md}
- Creates an images/ directory using Path("images").mkdir(exist_ok=True)
- For each spec in image_specs:
  * Gets placeholder, filename, out_path = images/filename
  * If out_path already exists, skip generation (do not regenerate)
  * Otherwise: try to call _gemini_generate_image_bytes with spec["prompt"] and
write the bytes to out_path
  * On any exception: replace the placeholder in md with a formatted error block
showing caption, alt, prompt, and error message in Markdown blockquote style
  * On success: replace placeholder with proper Markdown image:
![alt](images/filename) followed by *caption* on the next line
- Saves final md to disk as _safe_slug(blog_title).md
- Returns {"final": md}

Then complete the reducer subgraph:
- Add generate_and_place_images as a node called "generate_and_place_images"
- Add edges: START → merge_content → decide_images → generate_and_place_images →
END
- Compile the subgraph: reducer_subgraph = reducer_graph.compile()

After writing everything, explain: why does the code check if the image file
already exists before calling Gemini? What would happen to API costs if we removed
this check and always regenerated?
```

✅ What you should see after this step

- A _gemini_generate_image_bytes helper that calls Gemini and returns raw bytes
- A _safe_slug helper that produces clean filenames
- A generate_and_place_images function that loops through specs
- File existence check before generation — skips if already exists
- Graceful fallback error block when Gemini fails
- Placeholder replaced with proper Markdown image syntax on success
- reducer_subgraph compiled at the end
- Antigravity's explanation of API cost impact without the existence check

🚀 STEP 10

Wire the Main Graph + Runner Function

Five nodes, the reducer is now a compiled subgraph — run and test both modes

What's happening here?

The main graph in V4 has the exact same structure as V3: router → (conditional: research or orchestrator) → orchestrator → (fan-out) → workers → reducer → END.

The one structural change is that the 'reducer' node is no longer a simple function — it is the compiled reducer_subgraph from the previous steps. LangGraph allows a compiled subgraph to be used as a node in a parent graph. Internally, the subgraph runs its own three-node pipeline (merge_content → decide_images → generate_and_place_images) as a single atomic unit from the parent graph's perspective.

The runner function gains one new optional parameter: as_of. If not provided it defaults to today's date. All the new State fields (merged_md, md_with_placeholders, image_specs) are initialised to empty strings and lists.

Test it with an evergreen topic first to see no Tavily calls and image generation only. Then test with a current-events topic to see the full pipeline including research and image generation together.

⚡ What's improved vs V3 in this step

- reducer node is now reducer_subgraph (compiled subgraph as a node)
- Runner gains optional as_of parameter — defaults to today's date
- All new State fields pre-initialised in runner: merged_md, md_with_placeholders, image_specs
- Test: images/ folder created and .md file saved after run

YOUR PROMPT — Copy and paste into Antigravity

```
In bwa_backend.py, add the main graph and the runner function to complete the
backend.

Build the main graph:
- Create a StateGraph using the State TypedDict
- Add 5 nodes: router (router_node), research (research_node), orchestrator
(orchestrator_node), worker (worker_node), reducer (use reducer_subgraph — the
compiled subgraph from the previous step)
- Add edge from START to router
- Add conditional edges from router using route_next with dictionary format:
{"research": "research", "orchestrator": "orchestrator"}
- Add edge from research to orchestrator
- Add conditional edges from orchestrator using fanout targeting the "worker" node
- Add edge from worker to reducer
- Add edge from reducer to END
- Compile and store as app = g.compile()

Create a run(topic, as_of=None) function that:
- If as_of is None, sets it to date.today().isoformat()
- Calls app.invoke with all State fields pre-initialised: topic from argument, mode
as empty string, needs_research as False, queries as empty list, evidence as empty
list, plan as None, as_of from parameter, recency_days as 7, sections as empty
list, merged_md as empty string, md_with_placeholders as empty string, image_specs
as empty list, final as empty string
- Returns the full output
```

```
Test the agent with: run("The evolution of Small Language Models and the way
ahead")
After it runs:
1. What mode did the router choose?
2. Were any images generated? Check if images/ folder exists and contains PNG files
3. Does the .md file contain Markdown image syntax (![...](images/...))?
4. Print the value of out["final"] to see the full blog with images embedded
```

✅ **What you should see after this step**

- A StateGraph with 5 nodes — reducer is the compiled reducer_subgraph
- Graph wired identically to V3: router → conditional → research → orchestrator → fanout → workers → reducer → END
- A run() function with as_of parameter defaulting to today's date
- Agent runs without errors
- An images/ folder created with PNG files inside
- The .md file contains ![alt](images/filename) syntax
- out['final'] shows the complete blog with embedded image markdown

STEP 11

Frontend — Streamlit Web UI

Full web interface with 5 tabs, streaming progress, past blogs, and download buttons

What's happening here?

The frontend is a Streamlit app that wraps the entire backend in a web interface. It imports the compiled LangGraph app directly from bwa_backend.py and provides a clean UI for non-technical users to generate, preview, and download blogs without touching the terminal.

The sidebar has a topic text area, an as-of date picker, and a Generate Blog button. Below that is a Past Blogs section that automatically scans the current folder for .md files and lets you load any of them to preview in the UI.

The main area has 5 tabs. Plan tab shows the blog structure as a sortable dataframe — section titles, word counts, which sections need research/citations/code. Evidence tab shows the Tavily sources as a table. Markdown Preview tab renders the blog with embedded images (including locally saved PNG files). Images tab shows all generated images and offers a zip download. Logs tab shows the raw event stream from the LangGraph graph.

The app streams graph progress in real time using try_stream — showing which node is currently executing and a live JSON summary of the evolving state. Download buttons offer the markdown file alone or a full bundle zip containing the .md plus the images/ folder.

This file must be used exactly as provided — do not change a single line. Create it as bwa_frontend.py and run it with: streamlit run bwa_frontend.py

⚡ What's improved vs V3 in this step

- Brand new — frontend did not exist in V3
- 5 tabs: Plan, Evidence, Markdown Preview, Images, Logs
- Real-time streaming progress — shows which node is executing
- Past blogs sidebar — loads any previously generated .md file
- render_markdown_with_local_images — renders embedded PNG files from disk
- Download buttons: individual .md file or full bundle zip (md + images)

YOUR PROMPT — Copy and paste into Antigravity

```
Create a file called bwa_frontend.py in the same folder as bwa_backend.py.

This is a Streamlit web interface for the Blog Writer Agent. It imports the
compiled LangGraph app from bwa_backend and provides:
- A sidebar with a topic text area, an as-of date picker, and a Generate Blog
button
- A Past Blogs section in the sidebar that lists all .md files in the current
folder and lets the user load any of them
- A main area with 5 tabs: Plan (blog structure as a dataframe), Evidence (Tavily
sources as a table), Markdown Preview (renders the blog with embedded local
images), Images (shows generated PNGs and offers zip download), Logs (raw event
stream)
- Real-time streaming progress showing which node is currently running
- A live JSON summary of the evolving state while the graph runs
- Download buttons for the markdown file alone and for a full bundle zip containing
the .md plus the images/ folder
```

```
Do NOT write any code yourself. Instead, use EXACTLY the code from the Reference
Code box below. Copy it character-for-character into bwa_frontend.py.

To run the app after creating the file:
streamlit run bwa_frontend.py

The app will open in your browser at http://localhost:8501
```

REFERENCE CODE — Create bwa_frontend.py and paste this EXACTLY. Do not change a single character.

```
from __future__ import annotations

import json
import os
import re
import zipfile
from datetime import date
from io import BytesIO
from pathlib import Path
from typing import Any, Dict, Optional, List, Iterator, Tuple

import pandas as pd
import streamlit as st

from bwa_backend import app


def safe_slug(title: str) -> str:
    s = title.strip().lower()
    s = re.sub(r"[^a-z0-9 _-]+", "", s)
    s = re.sub(r"\s+", "_", s).strip("_")
    return s or "blog"


def bundle_zip(md_text: str, md_filename: str, images_dir: Path) -> bytes:
    buf = BytesIO()
    with zipfile.ZipFile(buf, "w", compression=zipfile.ZIP_DEFLATED) as z:
        z.writestr(md_filename, md_text.encode("utf-8"))
        if images_dir.exists() and images_dir.is_dir():
            for p in images_dir.rglob("*"):
                if p.is_file():
                    z.write(p, arcname=str(p))
    return buf.getvalue()


def images_zip(images_dir: Path) -> Optional[bytes]:
    if not images_dir.exists() or not images_dir.is_dir():
        return None
    buf = BytesIO()
    with zipfile.ZipFile(buf, "w", compression=zipfile.ZIP_DEFLATED) as z:
        for p in images_dir.rglob("*"):
            if p.is_file():
                z.write(p, arcname=str(p))
    return buf.getvalue()


def try_stream(graph_app, inputs: Dict[str, Any]) -> Iterator[Tuple[str, Any]]:
    try:
        for step in graph_app.stream(inputs, stream_mode="updates"):
```

```
            yield ("updates", step)
        out = graph_app.invoke(inputs)
        yield ("final", out)
        return
    except Exception:
        pass
    try:
        for step in graph_app.stream(inputs, stream_mode="values"):
            yield ("values", step)
        out = graph_app.invoke(inputs)
        yield ("final", out)
        return
    except Exception:
        pass
    out = graph_app.invoke(inputs)
    yield ("final", out)


def extract_latest_state(current_state, step_payload):
    if isinstance(step_payload, dict):
        if len(step_payload) == 1 and isinstance(next(iter(step_payload.values())), dict):
            inner = next(iter(step_payload.values()))
            current_state.update(inner)
        else:
            current_state.update(step_payload)
    return current_state


_MD_IMG_RE = re.compile(r"!\[(?P<alt>[^\]]*)\]\((?P<src>[^)]+)\)")
_CAPTION_LINE_RE = re.compile(r"^\*(?P<cap>.+)\*$")


def _resolve_image_path(src: str) -> Path:
    src = src.strip().lstrip("./")
    return Path(src).resolve()


def render_markdown_with_local_images(md: str):
    matches = list(_MD_IMG_RE.finditer(md))
    if not matches:
        st.markdown(md, unsafe_allow_html=False)
        return
    parts: List[Tuple[str, str]] = []
    last = 0
    for m in matches:
        before = md[last : m.start()]
        if before:
            parts.append(("md", before))
        alt = (m.group("alt") or "").strip()
        src = (m.group("src") or "").strip()
        parts.append(("img", f"{alt}|||{src}"))
        last = m.end()
    tail = md[last:]
    if tail:
        parts.append(("md", tail))
    i = 0
    while i < len(parts):
        kind, payload = parts[i]
        if kind == "md":
            st.markdown(payload, unsafe_allow_html=False)
            i += 1
            continue
        alt, src = payload.split("|||", 1)
```

```
        caption = None
        if i + 1 < len(parts) and parts[i + 1][0] == "md":
            nxt = parts[i + 1][1].lstrip()
            if nxt.strip():
                first_line = nxt.splitlines()[0].strip()
                mcap = _CAPTION_LINE_RE.match(first_line)
                if mcap:
                    caption = mcap.group("cap").strip()
                    rest = "\n".join(nxt.splitlines()[1:])
                    parts[i + 1] = ("md", rest)
        if src.startswith("http://") or src.startswith("https://"):
            st.image(src, caption=caption or (alt or None), use_container_width=True)
        else:
            img_path = _resolve_image_path(src)
            if img_path.exists():
                st.image(str(img_path), caption=caption or (alt or None),
use_container_width=True)
            else:
                st.warning(f"Image not found: `{src}` (looked for `{img_path}`)")
        i += 1


def list_past_blogs() -> List[Path]:
    cwd = Path(".")
    files = [p for p in cwd.glob("*.md") if p.is_file()]
    files.sort(key=lambda p: p.stat().st_mtime, reverse=True)
    return files


def read_md_file(p: Path) -> str:
    return p.read_text(encoding="utf-8", errors="replace")


def extract_title_from_md(md: str, fallback: str) -> str:
    for line in md.splitlines():
        if line.startswith("# "):
            t = line[2:].strip()
            return t or fallback
    return fallback


st.set_page_config(page_title="LangGraph Blog Writer", layout="wide")
st.title("Blog Writing Agent")

with st.sidebar:
    st.header("Generate New Blog")
    topic = st.text_area("Topic", height=120)
    as_of = st.date_input("As-of date", value=date.today())
    run_btn = st.button("🚀 Generate Blog", type="primary")
    st.divider()
    st.subheader("Past blogs")
    past_files = list_past_blogs()
    if not past_files:
        st.caption("No saved blogs found (*.md in current folder).")
        selected_md_file = None
    else:
        options: List[str] = []
        file_by_label: Dict[str, Path] = {}
        for p in past_files[:50]:
            try:
                md_text = read_md_file(p)
                title = extract_title_from_md(md_text, p.stem)
            except Exception:
```

```
                title = p.stem
            label = f"{title}  ·  {p.name}"
            options.append(label)
            file_by_label[label] = p
        selected_label = st.radio(
            "Select a blog to load", options=options, index=0,
label_visibility="collapsed"
        )
        selected_md_file = file_by_label.get(selected_label)
        if st.button("📂 Load selected blog"):
            if selected_md_file:
                md_text = read_md_file(selected_md_file)
                st.session_state["last_out"] = {
                    "plan": None, "evidence": [], "image_specs": [], "final": md_text,
                }
                st.session_state["topic_prefill"] = extract_title_from_md(md_text,
selected_md_file.stem)

if "topic_prefill" in st.session_state and isinstance(st.session_state["topic_prefill"],
str):
    pass

if "last_out" not in st.session_state:
    st.session_state["last_out"] = None

tab_plan, tab_evidence, tab_preview, tab_images, tab_logs = st.tabs(
    ["🧩 Plan", "🔎 Evidence", "📝 Markdown Preview", "🖼️ Images", "🧾 Logs"]
)

logs: List[str] = []


def log(msg: str):
    logs.append(msg)


if run_btn:
    if not topic.strip():
        st.warning("Please enter a topic.")
        st.stop()
    inputs: Dict[str, Any] = {
        "topic": topic.strip(), "mode": "", "needs_research": False, "queries": [],
        "evidence": [], "plan": None, "as_of": as_of.isoformat(), "recency_days": 7,
        "sections": [], "merged_md": "", "md_with_placeholders": "", "image_specs": [],
"final": "",
    }
    status = st.status("Running graph...", expanded=True)
    progress_area = st.empty()
    current_state: Dict[str, Any] = {}
    last_node = None
    for kind, payload in try_stream(app, inputs):
        if kind in ("updates", "values"):
            node_name = None
            if isinstance(payload, dict) and len(payload) == 1 and
isinstance(next(iter(payload.values())), dict):
                node_name = next(iter(payload.keys()))
            if node_name and node_name != last_node:
                status.write(f"➡️ Node: `{node_name}`")
                last_node = node_name
            current_state = extract_latest_state(current_state, payload)
            summary = {
                "mode": current_state.get("mode"),
                "needs_research": current_state.get("needs_research"),
```

```
                "queries": current_state.get("queries", [])[:5] if
isinstance(current_state.get("queries"), list) else [],
                "evidence_count": len(current_state.get("evidence", []) or []),
                "tasks": len((current_state.get("plan") or {}).get("tasks", [])) if
isinstance(current_state.get("plan"), dict) else None,
                "images": len(current_state.get("image_specs", []) or []),
                "sections_done": len(current_state.get("sections", []) or []),
            }
            progress_area.json(summary)
            log(f"[{kind}] {json.dumps(payload, default=str)[:1200]}")
        elif kind == "final":
            out = payload
            st.session_state["last_out"] = out
            status.update(label="✅ Done", state="complete", expanded=False)
            log("[final] received final state")

out = st.session_state.get("last_out")
if out:
    with tab_plan:
        st.subheader("Plan")
        plan_obj = out.get("plan")
        if not plan_obj:
            st.info("No plan found in output.")
        else:
            if hasattr(plan_obj, "model_dump"):
                plan_dict = plan_obj.model_dump()
            elif isinstance(plan_obj, dict):
                plan_dict = plan_obj
            else:
                plan_dict = json.loads(json.dumps(plan_obj, default=str))
            st.write("**Title:**", plan_dict.get("blog_title"))
            cols = st.columns(3)
            cols[0].write("**Audience:** " + str(plan_dict.get("audience")))
            cols[1].write("**Tone:** " + str(plan_dict.get("tone")))
            cols[2].write("**Blog kind:** " + str(plan_dict.get("blog_kind", "")))
            tasks = plan_dict.get("tasks", [])
            if tasks:
                df = pd.DataFrame([{
                    "id": t.get("id"), "title": t.get("title"),
                    "target_words": t.get("target_words"),
                    "requires_research": t.get("requires_research"),
                    "requires_citations": t.get("requires_citations"),
                    "requires_code": t.get("requires_code"),
                    "tags": ", ".join(t.get("tags") or []),
                } for t in tasks]).sort_values("id")
                st.dataframe(df, use_container_width=True, hide_index=True)
                with st.expander("Task details"):
                    st.json(tasks)
    with tab_evidence:
        st.subheader("Evidence")
        evidence = out.get("evidence") or []
        if not evidence:
            st.info("No evidence returned (maybe closed_book mode or no Tavily
key/results).")
        else:
            rows = []
            for e in evidence:
                if hasattr(e, "model_dump"):
                    e = e.model_dump()
                rows.append({"title": e.get("title"), "published_at":
e.get("published_at"),
                             "source": e.get("source"), "url": e.get("url")})
            st.dataframe(pd.DataFrame(rows), use_container_width=True, hide_index=True)
    with tab_preview:
```

```
        st.subheader("Markdown Preview")
        final_md = out.get("final") or ""
        if not final_md:
            st.warning("No final markdown found.")
        else:
            render_markdown_with_local_images(final_md)
            plan_obj = out.get("plan")
            if hasattr(plan_obj, "blog_title"):
                blog_title = plan_obj.blog_title
            elif isinstance(plan_obj, dict):
                blog_title = plan_obj.get("blog_title", "blog")
            else:
                blog_title = extract_title_from_md(final_md, "blog")
            md_filename = f"{safe_slug(blog_title)}.md"
            st.download_button("⬇️ Download Markdown", data=final_md.encode("utf-8"),
                               file_name=md_filename, mime="text/markdown")
            bundle = bundle_zip(final_md, md_filename, Path("images"))
            st.download_button("📦 Download Bundle (MD + images)", data=bundle,
                               file_name=f"{safe_slug(blog_title)}_bundle.zip",
mime="application/zip")
    with tab_images:
        st.subheader("Images")
        specs = out.get("image_specs") or []
        images_dir = Path("images")
        if not specs and not images_dir.exists():
            st.info("No images generated for this blog.")
        else:
            if specs:
                st.write("**Image plan:**")
                st.json(specs)
            if images_dir.exists():
                files = [p for p in images_dir.iterdir() if p.is_file()]
                if not files:
                    st.warning("images/ exists but is empty.")
                else:
                    for p in sorted(files):
                        st.image(str(p), caption=p.name, use_container_width=True)
                z = images_zip(images_dir)
                if z:
                    st.download_button("⬇️ Download Images (zip)", data=z,
                                       file_name="images.zip", mime="application/zip")
    with tab_logs:
        st.subheader("Logs")
        if "logs" not in st.session_state:
            st.session_state["logs"] = []
        if logs:
            st.session_state["logs"].extend(logs)
        st.text_area("Event log", value="\n\n".join(st.session_state["logs"][-80:]),
height=520)
else:
    st.info("Enter a topic and click **Generate Blog**.")
```

✅ What you should see after this step

- A bwa_frontend.py file created in the same folder as bwa_backend.py
- Running 'streamlit run bwa_frontend.py' opens the app in the browser
- The sidebar shows 'Generate New Blog' with topic input, date picker, and Generate button
- The sidebar shows 'Past blogs' with a list of .md files if any exist
- The main area has 5 tabs: Plan, Evidence, Markdown Preview, Images, Logs

- Clicking Generate Blog runs the agent and shows live streaming progress
- After completion, the Plan tab shows a dataframe of sections
- The Markdown Preview tab renders the blog with embedded images
- Download buttons appear for .md and bundle zip

How to Get Your Google API Key

You need a Google API key to use Gemini 2.5 Flash for image generation. Get it for free from Google AI Studio in under 2 minutes.

Step-by-step: Get your key

1. Go to https://aistudio.google.com in your browser and sign in with your Google account
2. Click 'Get API Key' in the left sidebar, then click 'Create API Key'
3. Copy the key — it starts with AIza
4. Paste it into your .env file: GOOGLE_API_KEY=AIza-your-key-here
5. The free tier of Google AI Studio gives you generous usage — more than enough for this project

🔗 **Sign up at: https://aistudio.google.com**

Frequently Asked Questions

Question	Answer
Do I need a credit card?	No. Google AI Studio's free tier requires only a Google account.
Which Gemini model generates images?	gemini-2.5-flash-image — a fast, capable image generation model that returns raw PNG bytes.
What if image generation fails?	The agent handles this gracefully. Failed images are replaced with an error block in the blog — the blog is still fully readable.
How many images can I generate for free?	Google AI Studio's free tier is generous. For this project (max 3 images per blog run) you can run the agent many times without hitting limits.
Can I use a different image model?	Yes — you could swap Gemini for DALL-E 3 via OpenAI's API. You would need to replace _gemini_generate_image_bytes with an OpenAI images.generate() call and adjust the byte extraction.

⚠️ **All three keys must be in your .env file**

```
OPENAI_API_KEY=sk-your-openai-key-here
TAVILY_API_KEY=tvly-your-tavily-key-here
GOOGLE_API_KEY=AIza-your-google-key-here
```

Never share your .env file or commit it to GitHub. Add .env to your .gitignore file.

🎉

You built a complete AI-powered blog platform.

Here is what you actually learned in V4:

🎨 Image planning — how an LLM decides WHERE in a document images help most

🖼️ Gemini image generation — calling an AI vision model to produce PNG bytes from a prompt

🔀 Subgraphs — using a compiled LangGraph as a node inside a larger graph

🛡️ Graceful fallback — production systems never crash on a single failed image

📅 Date-aware pipelines — filtering evidence by recency to prevent stale citations

🌐 Streamlit UI — wrapping an AI backend in a shareable web interface

📦 Bundle downloads — packaging markdown + images into a single zip for publishing

Next challenge: Add a human-in-the-loop step where the user can approve or edit the image plan before Gemini generates anything. What new node would you add, and where in the subgraph would it go?